



Minimum spanning tree

Martedì 12 maggio 2026

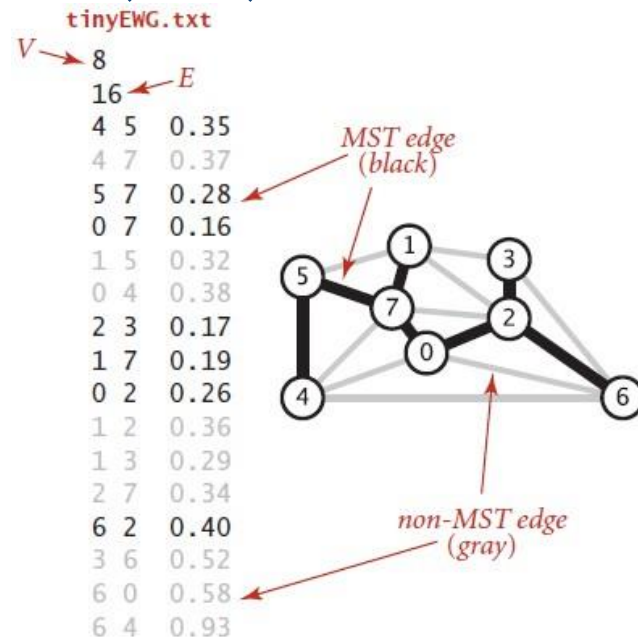
Grafo pesato

- Un **Grafo pesato** (edge-weighted graph) è un un grafo non orientato in cui ogni arco ha un peso (o costo) associato.

applicazione	vertice	arco
circuito	componente	connessione
trasporto	aeroporto	linea aerea
energia	centrale	linea di trasmissione
immagine	feature	relazione di prossimità

Minimo albero ricoprente: 1926

- Dato un grafo pesato (connesso) G , un albero ricoprente (spanning tree) di G è un sottografo T connesso e aciclico (un albero) che include tutti i vertici di G . Lo spanning tree che ha minima somma dei pesi agli archi si chiama minimum spanning tree (MST).

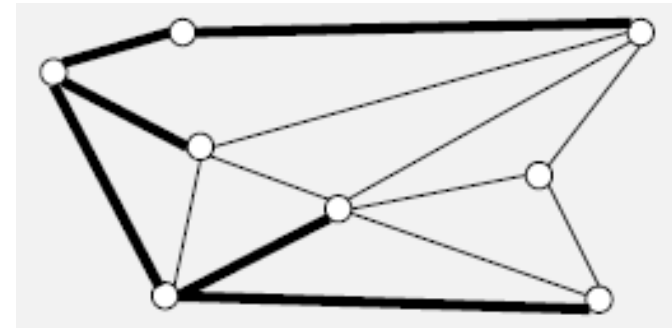
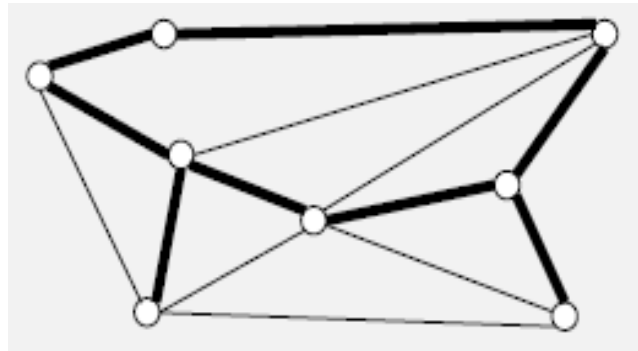
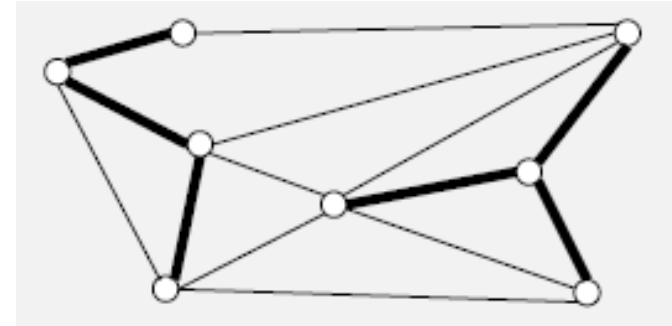
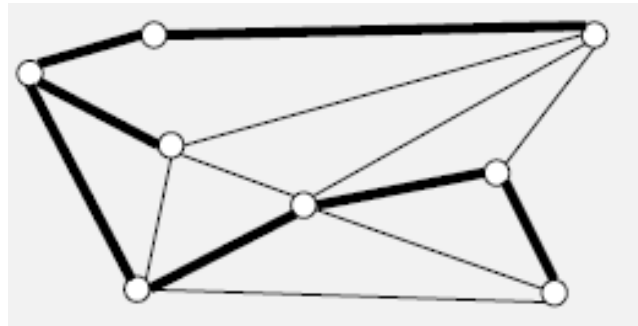


Assumiamo che:

- Il grafo sia connesso, altrimenti possiamo parlare di minimum spanning forest
- I pesi associati agli archi possono anche assumere valore nulli o negativi

Esempi

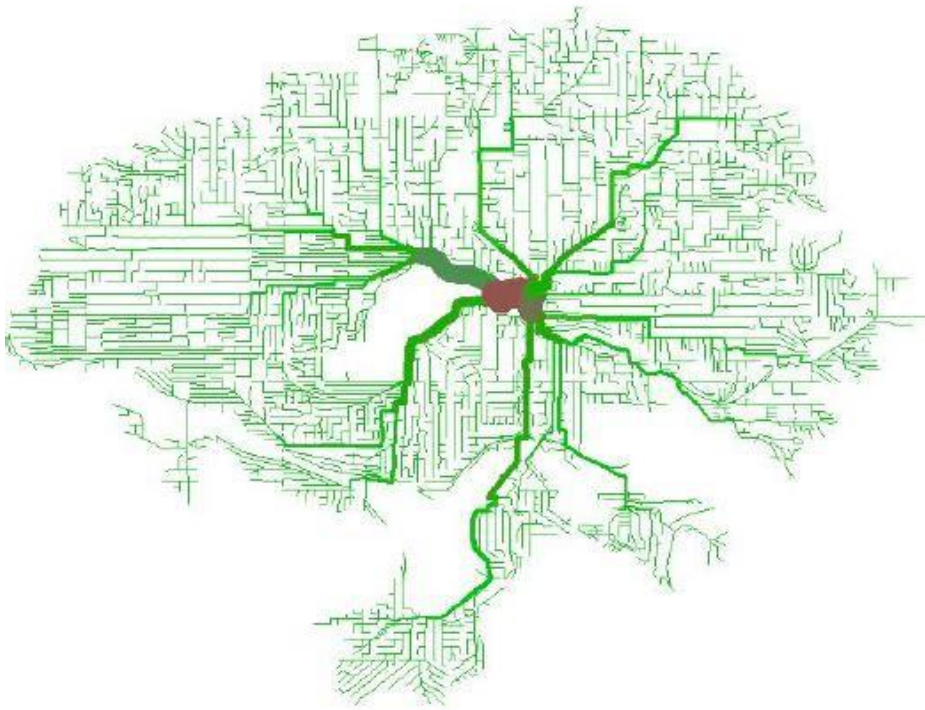
- Quali tra i seguenti sono spanning tree? E Perché?



Alcune applicazioni

Progettazione di network

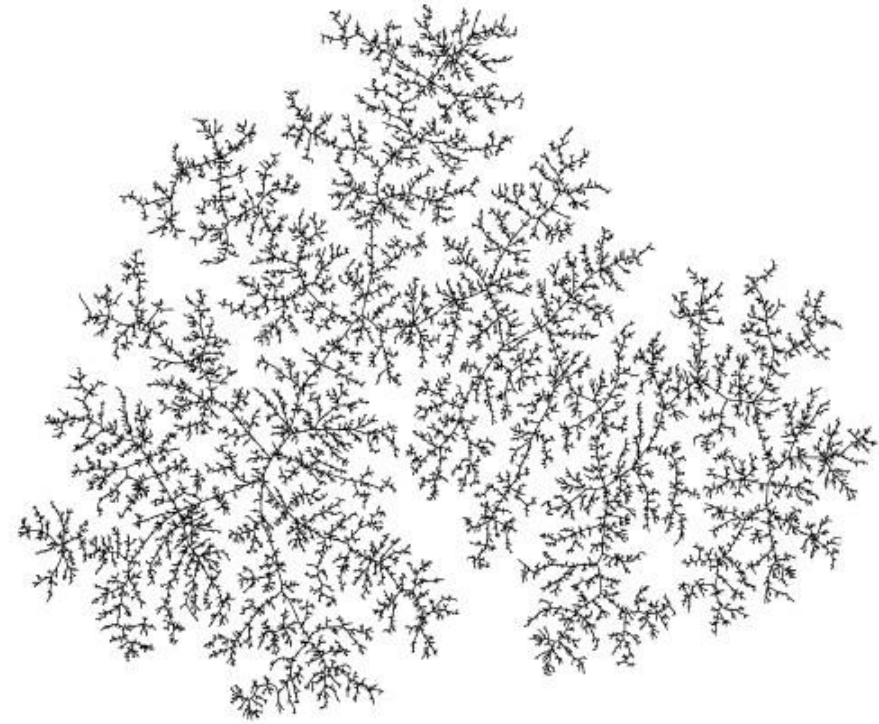
MST of bicycle routes in North Seattle



<http://www.flickr.com/photos/ewedistrict/21980840>

Modelli naturali

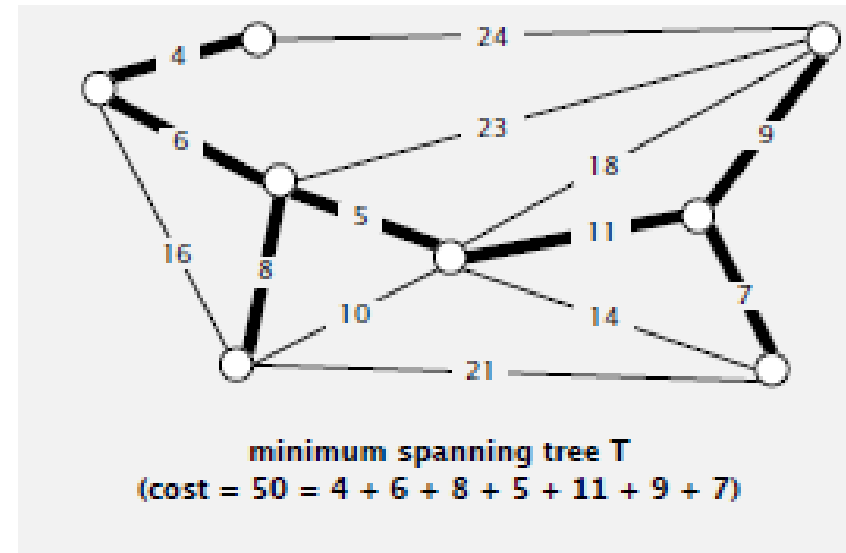
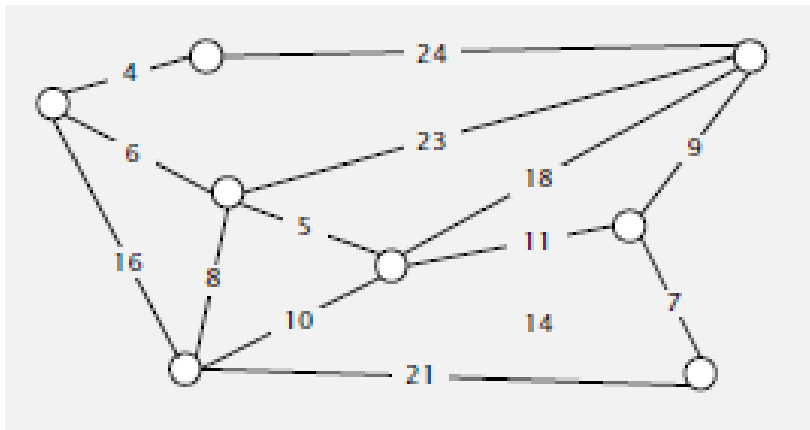
MST of random graph



<http://algo.inria.fr/broutin/gallery.html>

Esempio di MST

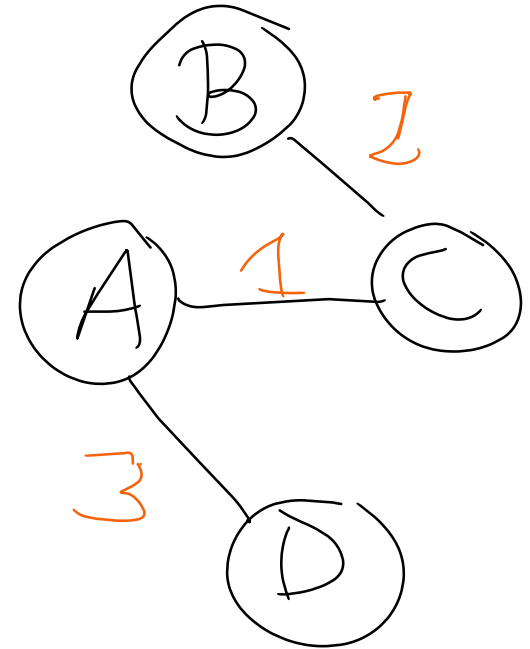
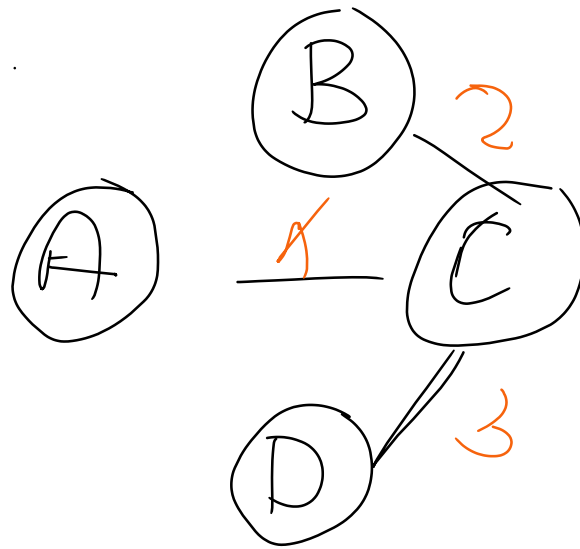
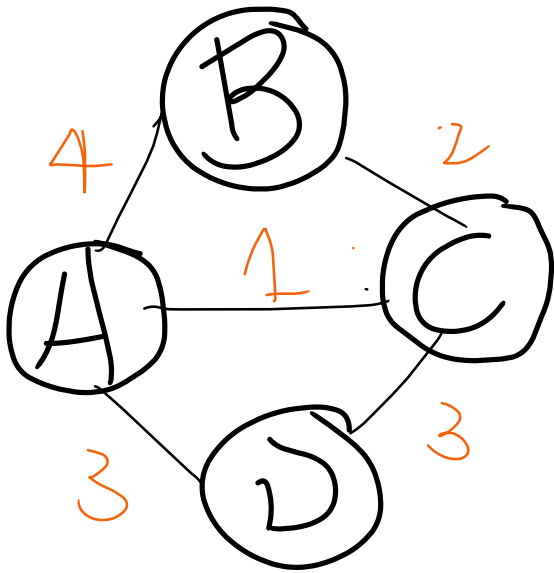
- Dato un grafo pesato, trovare un MST.



- Quanti sono i MST in un grafo? Come trovare tutti i MST di un grafo?

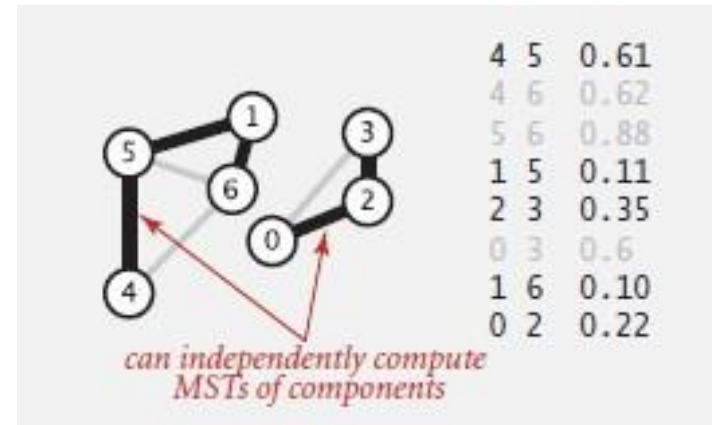
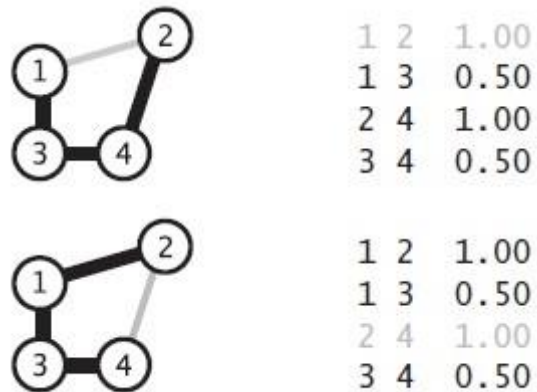
Esempi di grafo

□ Esempio di grafo e dei suoi minimi alberi ricoprenti



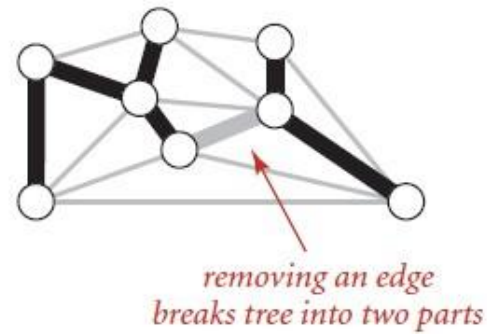
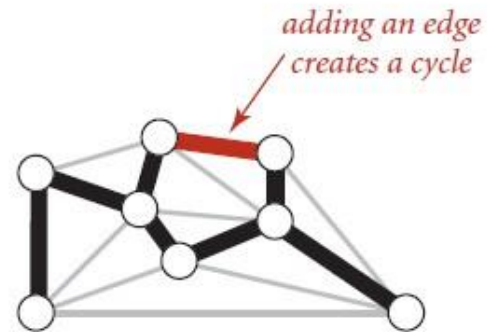
Unicità del MST

- Supponiamo per semplicità che il grafo di partenza sia connesso e che i pesi degli archi siano tutti diversi
- La conseguenza è che il MST è unico.



Proprietà dell'albero

- Aggiungere un arco crea un ciclo
- Rimuovere un arco rimuove la connessione



Tecnica Greedy: Taglio

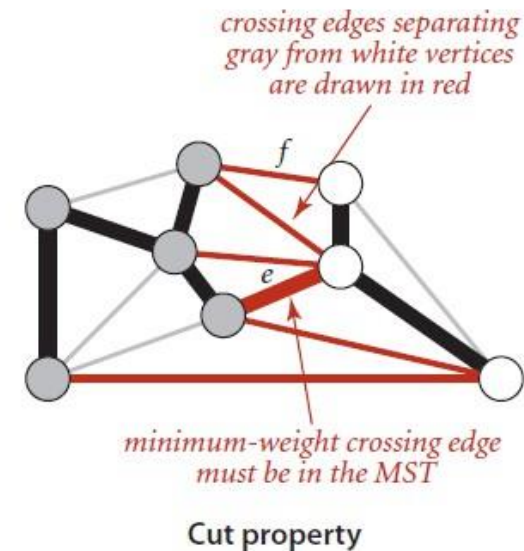
- Un taglio (cut) di un grafo è una partizione dei suoi vertici in due insiemi non vuoti;
- Dato un taglio, un arco che attraversa il taglio (crossing edge) è un arco che connette un vertice di un insieme con un vertice dell'altro insieme

Proprietà dei MST

- **REGOLA DEL TAGLIO** Scegli un taglio in G che non contiene archi neri, tra tutti gli archi non colorati seleziona un arco di costo minimo e coloralo
- **REGOLA DEL CICLO** Scegli un ciclo semplice in G , che non contiene archi rossi, tra tutti gli archi non colorati del ciclo seleziona un arco di costo massimo e coloralo di rosso

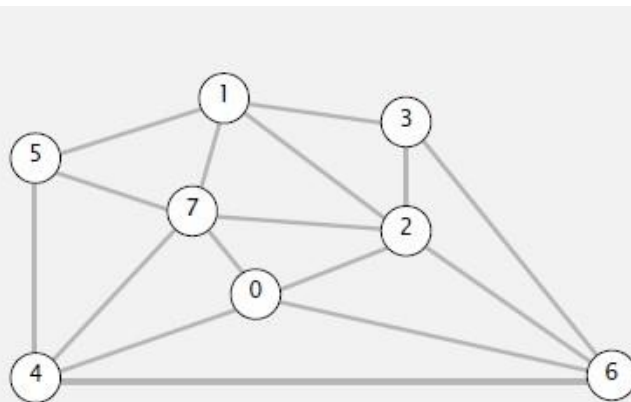
Proprietà del taglio

- **TEOREMA:** Dato un qualsiasi taglio in un grafo, il crossing edge di peso minimo sta nel MST.
- **Dimostrazione:** Per assurdo, si supponga che il crossing edge di minor peso **e** non stia nel MST.
 - Consideriamo il grafo costituito da **e** e dal MST, esso ha un ciclo;
 - Tale ciclo deve contenere almeno un altro crossing edge **f**;
 - **f** deve avere peso maggiore di **e**;
 - sostituendo **f** con **e** si otterrebbe uno spanning tree di peso inferiore.



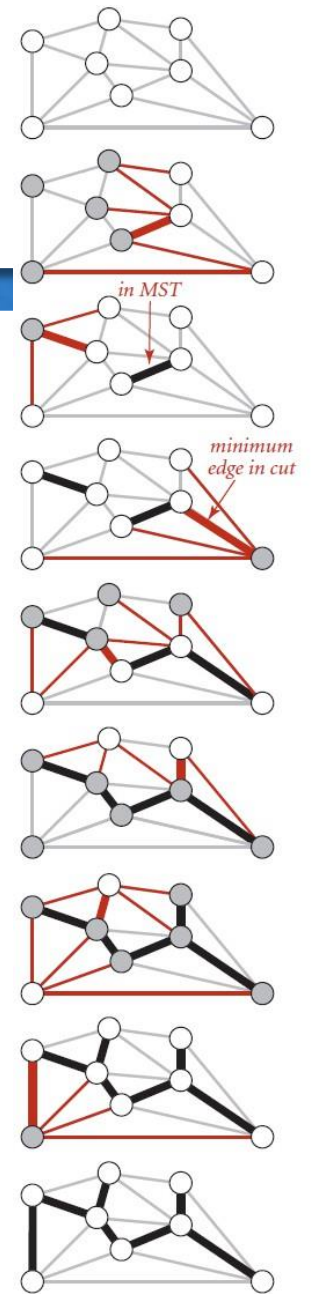
MST: algoritmo greedy

1. Colora tutti gli archi di grigio;
2. Trova un qualsiasi cut privo di crossing edge neri. Colora di nero il crossing edge di peso minimo;
3. Ripeti (2) fino a colorare $|V|-1$ archi di nero.



an edge-weighted graph

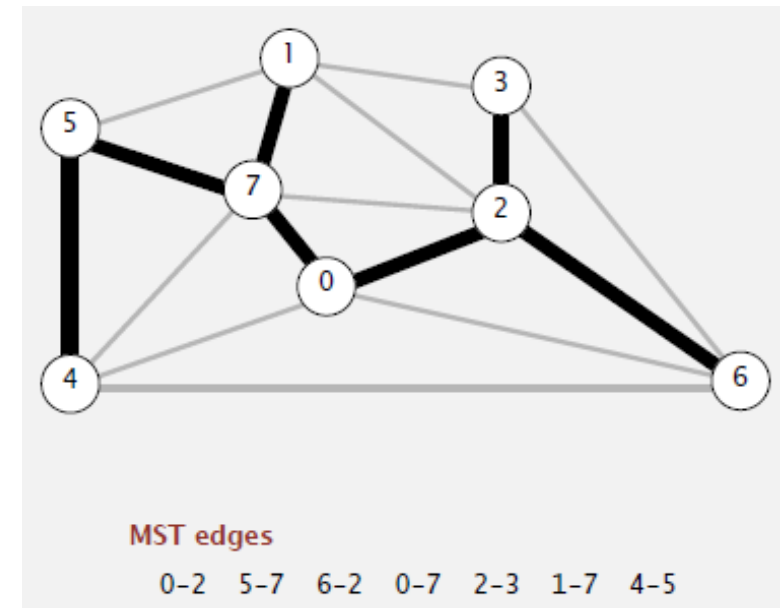
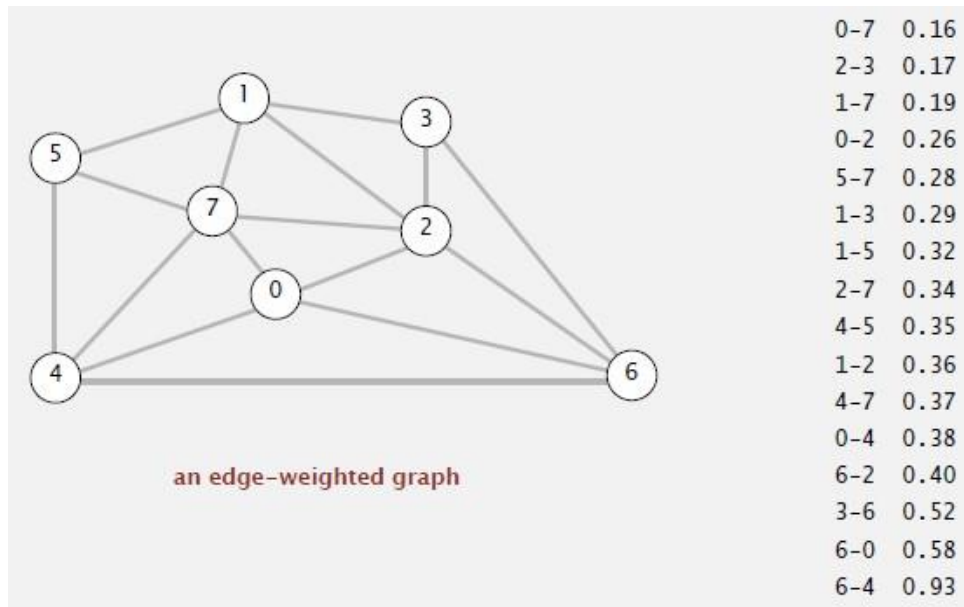
0-7	0.16
2-3	0.17
1-7	0.19
0-2	0.26
5-7	0.28
1-3	0.29
1-5	0.32
2-7	0.34
4-5	0.35
1-2	0.36
4-7	0.37
0-4	0.38
6-2	0.40
3-6	0.52
6-0	0.58
6-4	0.93



Greedy MST algorithm

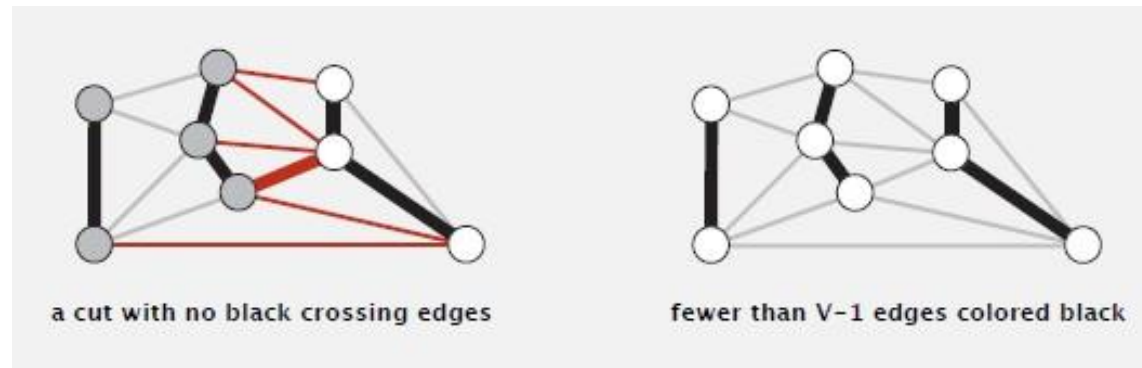
MST: algoritmo greedy

1. Colora tutti gli archi di grigio;
2. Trova un qualsiasi cut privo di crossing edge neri. Colora di nero il crossing edge di peso minimo;
3. Ripeti (2) fino a colorare $|V|-1$ archi di nero.



Algoritmo greedy: correttezza

- Ogni arco colorato di nero sta nel MST (proprietà del taglio);
- Se ci sono meno di $|V|-1$ archi neri, allora esiste un cut privo di archi neri.
- Una volta che si trovano $|V|-1$ archi neri, essi formano uno spanning tree



- Nota: Funziona anche se gli archi non sono tutti distinti. Se il grafo non è connesso restituisce una minimum spanning forest (MST di ogni componente connessa)
- Diversi algoritmi per il calcolo del MST si caratterizzano per la scelta del cut e per il modo in cui estraggono il crossing edge a peso minimo.



Implementazione di un grafo pesato

Quale rappresentazione

- Usando la matrice di adiacenza, essa può contenere i pesi degli archi invece dei valori 1 e 0 (per grafi semplici);
- Usando una lista delle adiacenze, possiamo definire un nodo che contiene sia un vertice che un campo "peso" da inserire nella lista.
- E' possibile definire una classe per rappresentare gli archi.

Una classe per gli archi

```
public class Edge implements Comparable<Edge>
```

Edge(int v, int w, double weight)	<i>initializing constructor</i>
double weight()	<i>weight of this edge</i>
int either()	<i>either of this edge's vertices</i>
int other(int v)	<i>the other vertex</i>
int compareTo(Edge that)	<i>compare this edge to e</i>
String toString()	<i>string representation</i>

- Elaborazione degli archi
int v = e.either(), w = e.other(v);

Archi pesati: implementazione

```
public class Edge implements Comparable<Edge> {  
    // immutable edge  
    private final int v, w;  
    private final double weight;  
  
    public Edge(int v, int w, double weight) {  
        this.v = v;  
        this.w = w;  
        this.weight = weight;  
    }  
  
    public int either() {  
        return v;  
    }  
  
    public int other(int vertex) {  
        if (vertex == v) return w;  
        else if (vertex == w) return v;  
        else throw new IllegalArgumentException("Illegal_endpoint");  
    }  
  
    public int compareTo(Edge that) {  
        return Double.compare(this.weight, that.weight);  
    }  
}
```

Una classe per i grafi pesati

```
public class EdgeWeightedGraph
```

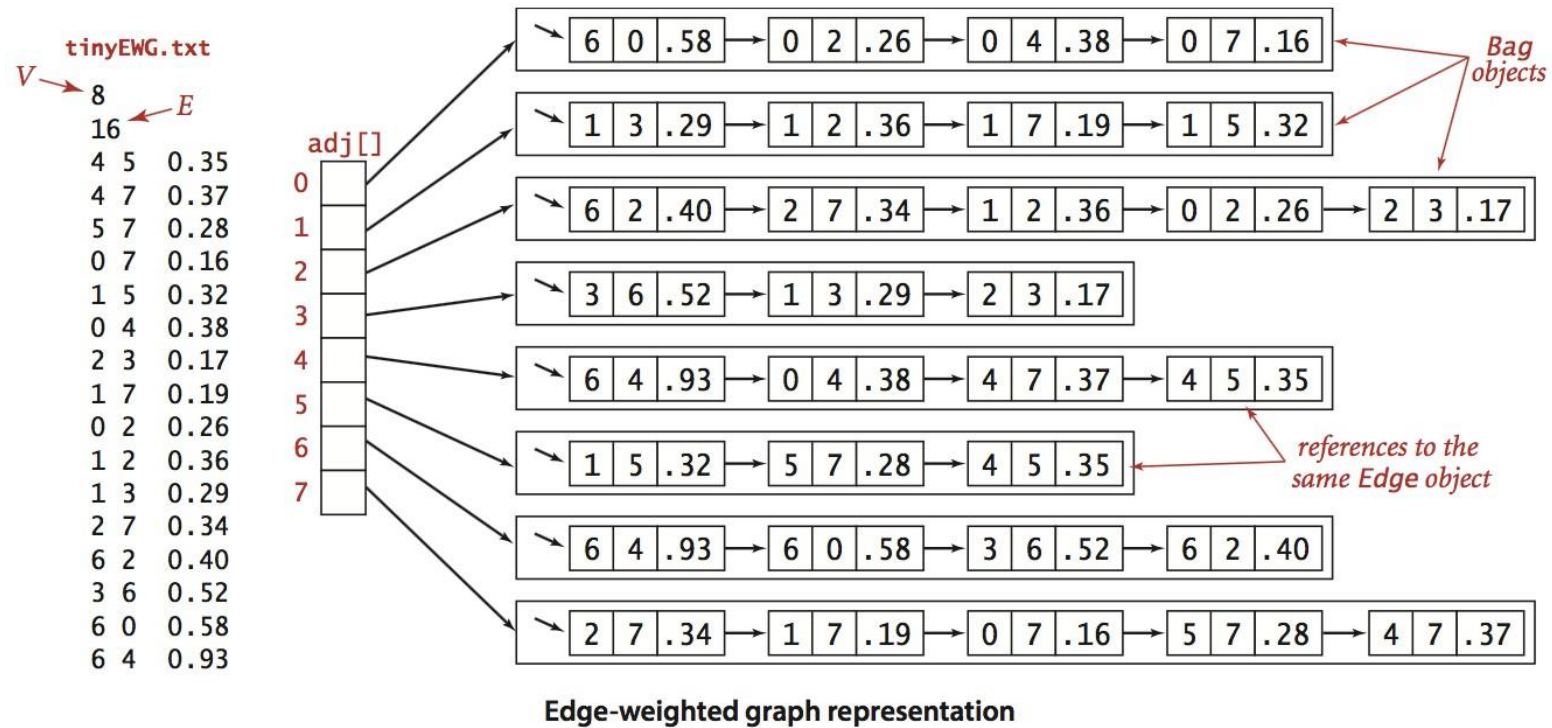
EdgeWeightedGraph(int V)	<i>create an empty V-vertex graph</i>
EdgeWeightedGraph(In in)	<i>read graph from input stream</i>
int V()	<i>number of vertices</i>
int E()	<i>number of edges</i>
void addEdge(Edge e)	<i>add edge e to this graph</i>
Iterable<Edge> adj(int v)	<i>edges incident to v</i>
Iterable<Edge> edges()	<i>all of this graph's edges</i>
String toString()	<i>string representation</i>

- I self-loops e gli archi paralleli sono consentiti.

Il metodo edges()

```
public Iterable<Edge> edges()
{
    Bag<Edge> b = new Bag<Edge>();
    for (int v = 0; v < V; v++)
        for (Edge e : adj[v])
            if (e.other(v) > v) b.add(e);
    return b;
}
```

Esempio



Grafi pesati: implementazione

```
public class EdgeWeightedGraph {
    private final int V;
    private int E;
    private Bag<Edge>[] adj;

    public EdgeWeightedGraph(int V) {
        this.V = V;
        this.E = 0;
        adj = (Bag<Edge>[]) new Bag[V];
        for (int v = 0; v < V; v++)
            adj[v] = new Bag<Edge>();
    }

    public void addEdge(Edge e) {
        int v = e.either(), w = e.other(v);
        adj[v].add(e);
        adj[w].add(e);
        E++;
    }

    public Iterable<Edge> adj(int v) {
        return adj[v];
    }
}
```


Una classe per MST

```
public class MST
```

```
    MST(EdgeWeightedGraph G)
```

constructor

```
    Iterable<Edge> edges()
```

all of the MST edges

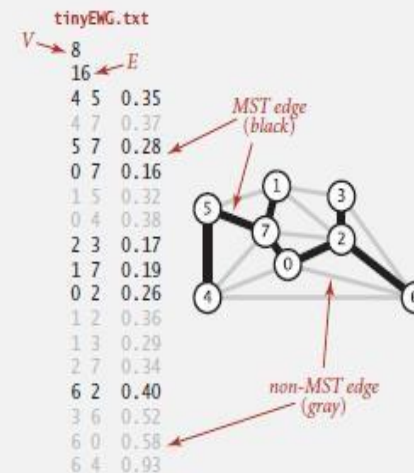
```
    double weight()
```

weight of MST

```
public static void main(String[] args)
{
    In in = new In(args[0]);
    EdgeWeightedGraph G;
    G = new EdgeWeightedGraph(in);

    MST mst = new MST(G);
    for (Edge e : mst.edges())
        StdOut.println(e);
    StdOut.println(mst.weight());
}
```

MST test client

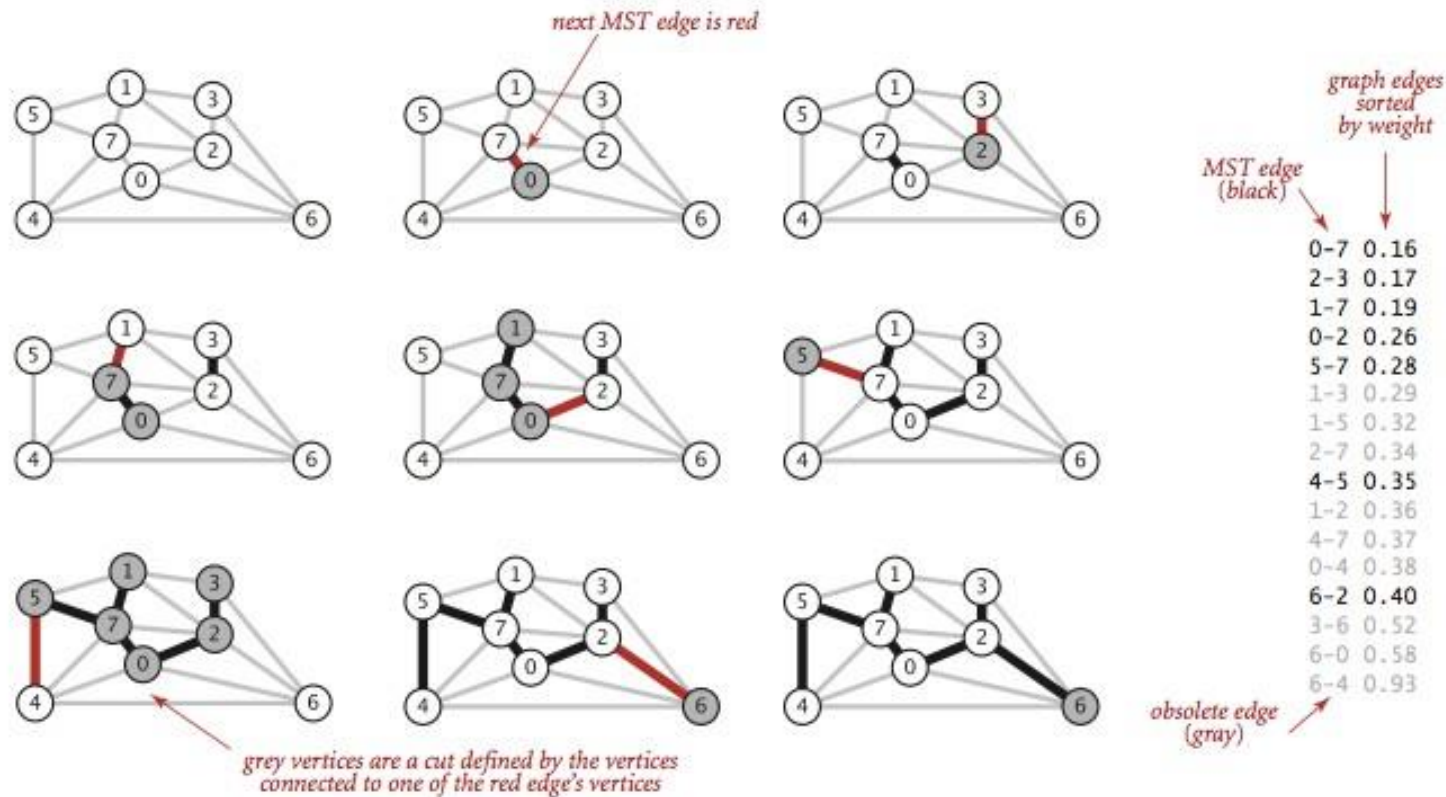


```
% java MST tinyEWG.txt
0-7 0.16
1-7 0.19
0-2 0.26
2-3 0.17
5-7 0.28
4-5 0.35
6-2 0.40
1.81
```

Algoritmo di Kruskal (1956)

1. Considera gli archi in ordine crescente di peso;
2. Aggiungi un arco all'albero se così facendo non introduci un ciclo, altrimenti scarta l'arco.
3. Ripeti (2) fino ad aver inserito $|V| - 1$ archi nell'albero.

Esempio di esecuzione



Altro esempio di esecuzione: peso=distanza euclidea

Algoritmo di Kruskal: correttezza

- TEOREMA: L'algoritmo di Kruskal trova il MST
- Dimostrazione: E' un caso speciale dell'algoritmo greedy:
 - sia $e = v-w$ l'arco colorato di nero;
 - si consideri per il cut l'insieme di vertici connessi a v attraverso archi dell'albero T ;
 - nessun crossing edge è nero;
 - nessun crossing edge ha peso minore (sono scelti in ordine crescente).

Algoritmo di Kruskal: implementazione

- **Problema:** come scegliere il prossimo arco meno pesante?

Soluzioni:

sorting degli archi, oppure priority queue (heap).

- **Problema:** l'arco $e = v - w$ forma un ciclo?

Soluzioni:

- Usare DFS da v sulla foresta T fin qui formata e vedere se w è raggiungibile: $|V| - 1$ passi al massimo.
- Mantenere una struttura union-find e verificare se v e w sono connessi: $\log V$ passi al massimo:
 1. mantieni un set per ogni componente connessa in T ;
 2. se v e w stanno nello stesso set non aggiungere l'arco (creerebbe un ciclo);
 3. aggiungi l'arco $v - w$ a T unendo i due set dei suoi estremi.

Algoritmo di Kruskal: implementazione

```
public class KruskalMST {
    private double weight;           // weight of MST
    private Queue<Edge> mst = new Queue<Edge>(); // edges in MST

    public KruskalMST(EdgeWeightedGraph G) {
        MinPQ<Edge> pq = new MinPQ<Edge>();
        for (Edge e : G.edges()) {
            pq.insert(e);
        }

        UF uf = new UF(G.V());
        while (!pq.isEmpty() && mst.size() < G.V() - 1) {
            Edge e = pq.delMin();
            int v = e.either();
            int w = e.other(v);
            if (!uf.connected(v, w)) { // v-w does not create a cycle
                uf.union(v, w); // merge v and w components
                mst.enqueue(e); // add edge e to mst
                weight += e.weight();
            }
        }
    }

    public Iterable<Edge> edges() {
        return mst;
    }

    public double weight() {
        return weight;
    }
}
```

Algoritmo di Kruskal: complessità

□ TEOREMA: L'algoritmo di Kruskal trova il MST in tempo proporzionale a $E \log E$ (caso peggiore) e spazio proporzionale a E .

□ Dimostrazione:

Ordinamento degli archi: Ordina tutti gli archi del grafo in ordine crescente di peso. Inizializzazione: Crea una struttura Disjoint-Set (Union-Find) per tenere traccia delle componenti connesse, inizialmente ponendo ogni nodo in un insieme separato. Ciclo principale: Itera attraverso gli archi ordinati e, per ogni arco (u,v) controlla se u e v appartengono a componenti connesse diverse. Se no, aggiunge l'arco all'MST e unisce le componenti.

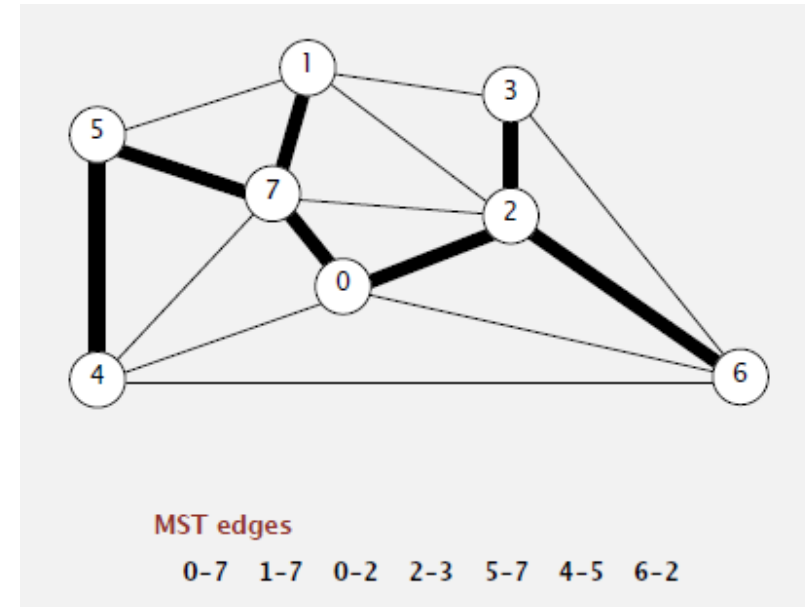
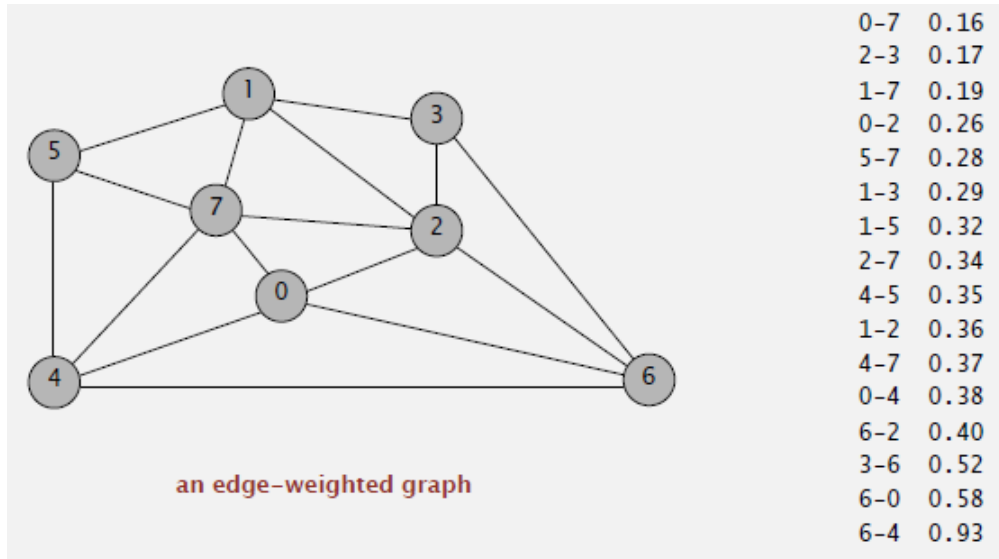
<i>operazione</i>	<i>frequenza</i>	<i>tempo per operazione</i>
<i>costruisci PQ</i>	1	$E \log E$
<i>delete-min</i>	E	$\log E$
<i>union</i>	V	$\log V$
<i>connected</i>	E	$\log V$

Algoritmo di Prim

(Jarník 1930, Dijkstra 1957, Prim 1959)

1. Parti dal vertice o e comincia a far crescere l'albero T ;
2. Aggiungi all'albero l'arco di minor peso tra quelli che hanno un estremo in T e l'altro estremo non in T ;
1. Ripeti (2) fino ad aver inserito $|V|-1$ archi nell'albero.

Esempio



□ Altro esempio di esecuzione: peso=distanza euclidea

Algoritmo di Prim: correttezza

- TEOREMA: L'algoritmo di Prim trova il MST
- E' un caso speciale dell'algoritmo greedy:
 - sia e l'arco di minor peso tra quelli che hanno un estremo in T e l'altro estremo non in T ;
 - si consideri per il cut l'insieme di vertici connessi nell'albero T ;
 - nessun crossing edge è nero;
 - nessun crossing edge ha peso minore di e (sono scelti in ordine crescente).

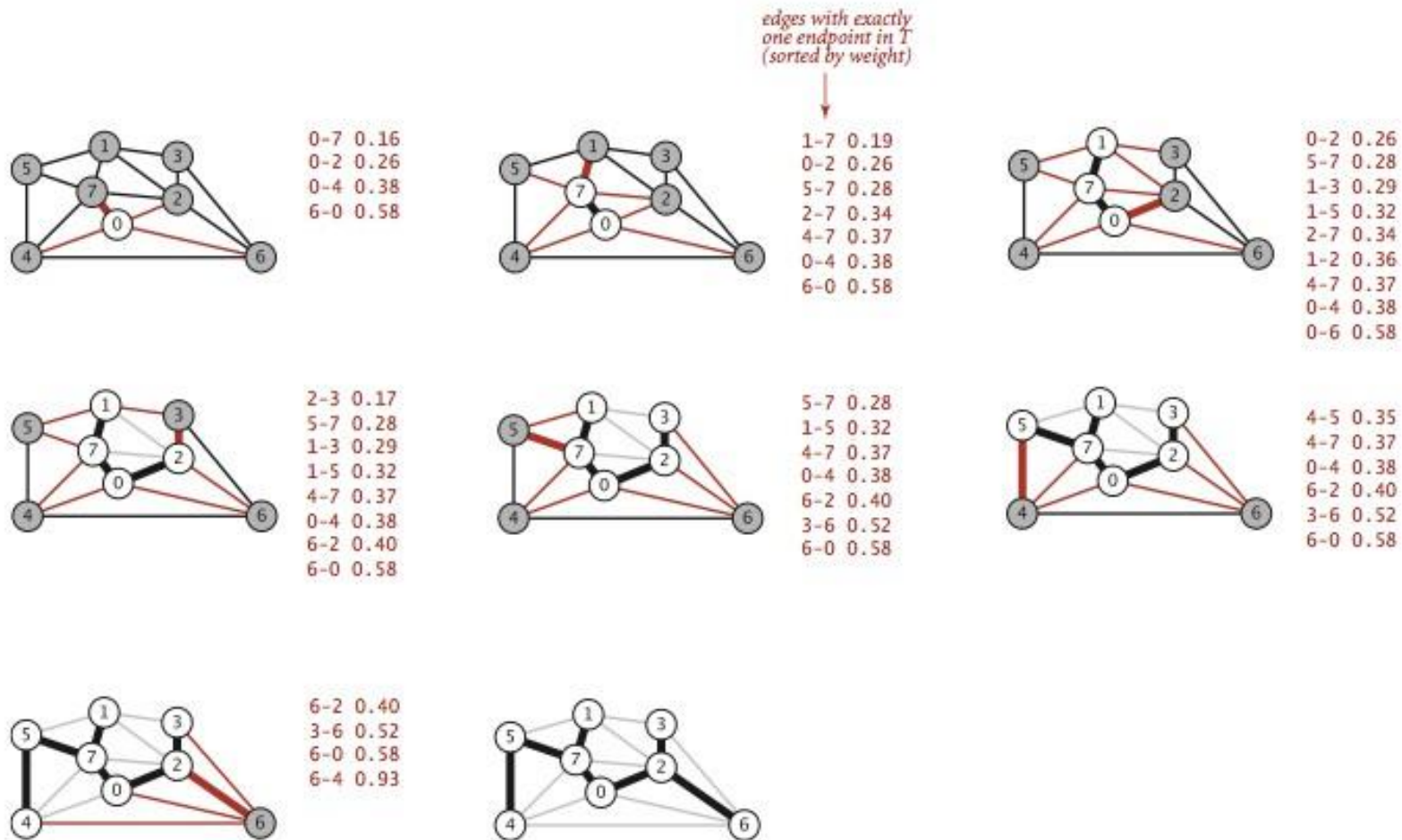
Implementazione

- *Vertici sull'albero*: Un vertex-indexed array di boolean `marked[]`, dove `marked[v]` è true se `v` appartiene all'albero.
- *Archi sull'albero*: una queue mst per memorizzare gli archi del MST o un vertex-indexed array `edgeTo[]` di oggetti `Edge`, dove `edgeTo[v]` è l' `Edge` che connette `v` al tree.
- *Crossing edges*: si usa una `MinPQ<Edge>` coda a priorità che confronta gli archi per peso.
- **Problema**: come trovare l'arco di minor peso tra quelli che hanno un estremo in `T` e l'altro estremo non in `T`?
Soluzione:
 - con priority queue (heap), in $\log E$ passi.

Algoritmo di Prim: versione lazy

- **Lazy Prim:** mantieni una PQ degli archi che hanno almeno un estremo in T .
 - delete-min per estrarre l'arco $e = v - w$ da aggiungere a T ;
 - scarta e se i suoi estremi sono entrambi in T (gli archi doppiamente marcati non vengono espulsi dalla heap -> laziness),
 - Altrimenti
 - se w è l'estremo non marcato di e , aggiungi a PQ tutti gli archi ad esso incidenti, con altro estremo non in T ;
 - aggiungi e a T e marca w .

Esempio



Implementazione

```
public class LazyPrimMST {
    private double weight;           // total weight of MST
    private Queue<Edge> mst;         // edges in the MST
    private boolean[] marked;       // marked[v] = true if v on tree
    private MinPQ<Edge> pq;         // edges with one endpoint in tree

    public LazyPrimMST(EdgeWeightedGraph G) {
        mst = new Queue<Edge>();
        pq = new MinPQ<Edge>();
        marked = new boolean[G.V()];
        for (int v = 0; v < G.V(); v++) // run Prim from all vertices to
            if (!marked[v]) prim(G, v); // get a minimum spanning forest
    }

    private void prim(EdgeWeightedGraph G, int s) {
        scan(G, s);
        while (!pq.isEmpty()) {
            Edge e = pq.delMin();
            int v = e.either(), w = e.other(v);
            if (marked[v] && marked[w]) continue; // laziness
            mst.enqueue(e);
            weight += e.weight();
            if (!marked[v]) scan(G, v); // v becomes part of tree
            if (!marked[w]) scan(G, w); // w becomes part of tree
        }
    }
    ...
}
```

Implementazione

```
...  
    // add all edges e incident to v onto pq  
    // if the other endpoint has not yet been scanned  
    private void scan(EdgeWeightedGraph G, int v) {  
        marked[v] = true;  
        for (Edge e : G.adj(v))  
            if (!marked[e.other(v)]) pq.insert(e);  
    }  
  
    public Iterable<Edge> edges() {  
        return mst;  
    }  
  
    public double weight() {  
        return weight;  
    }  
}
```

Lazy Prim: complessità

- **TEOREMA:** L'algoritmo di Prim nell'implementazione lazy trova il MST in tempo proporzionale a $E \log E$ (caso peggiore), con spazio proporzionale a E .
- **Dimostrazione:** Inserimento nella Coda, Estrazione Minimo, operazione sui nodi.

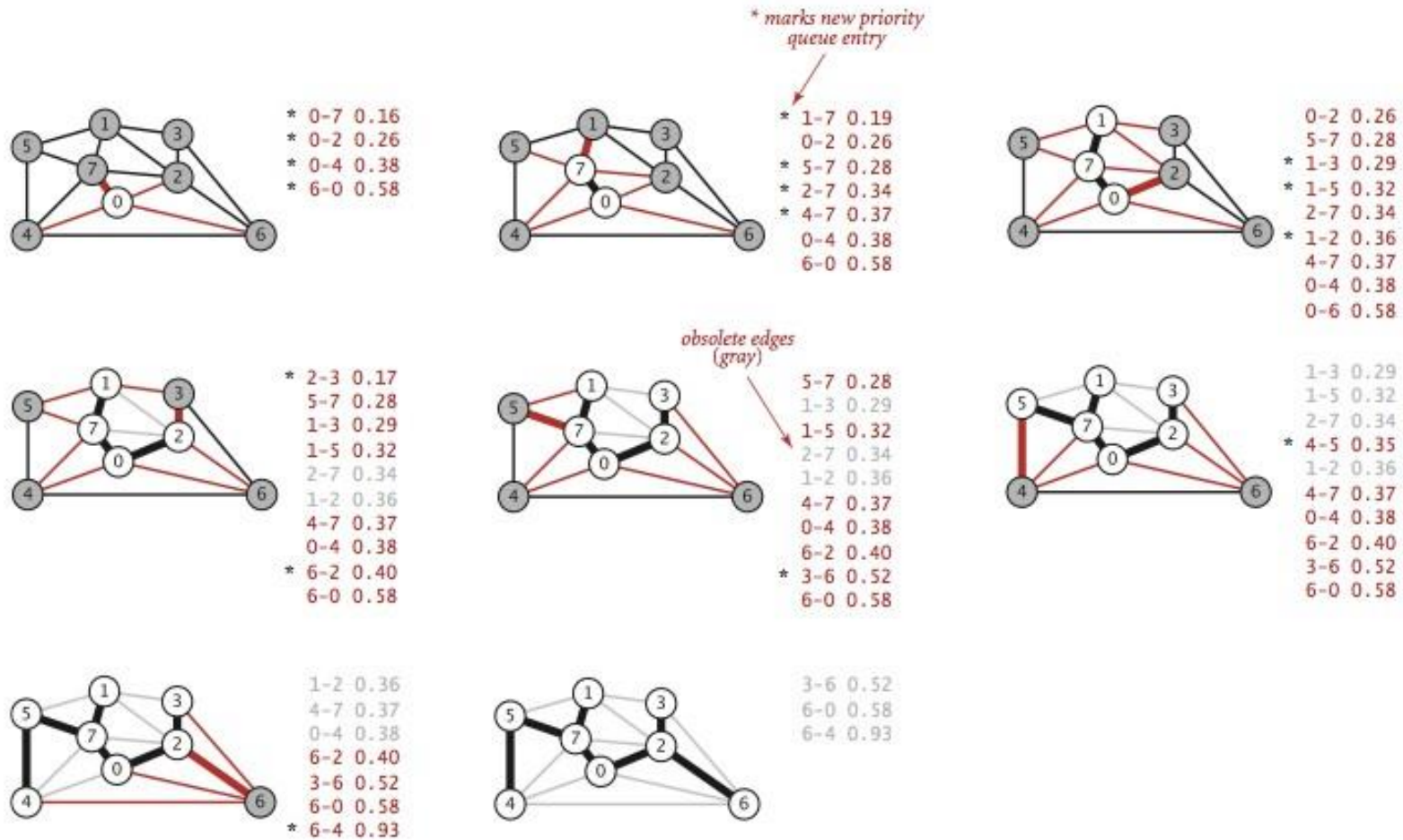
Operazioni sui Nodi: Poiché ogni arco viene inserito e rimosso al massimo una volta, il numero totale di operazioni sulla coda di priorità è proporzionale a E .

<i>operazione</i>	<i>frequenza</i>	<i>tempo per operazione</i>
<i>delete-min</i>	E	$\log E$
<i>insert</i>	E	$\log E$

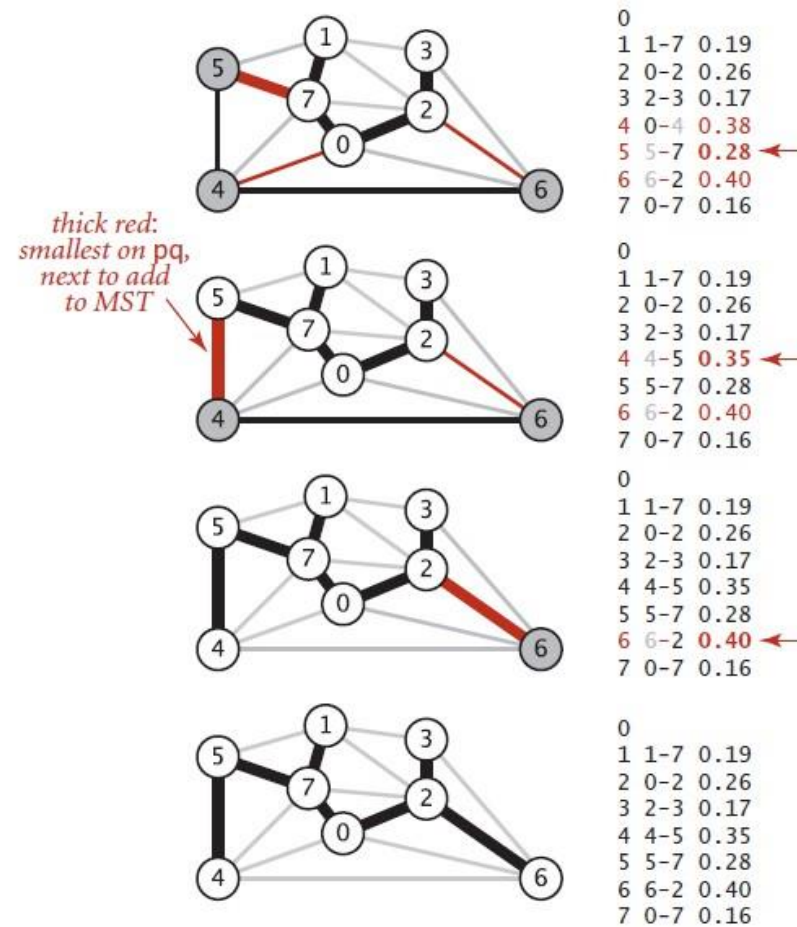
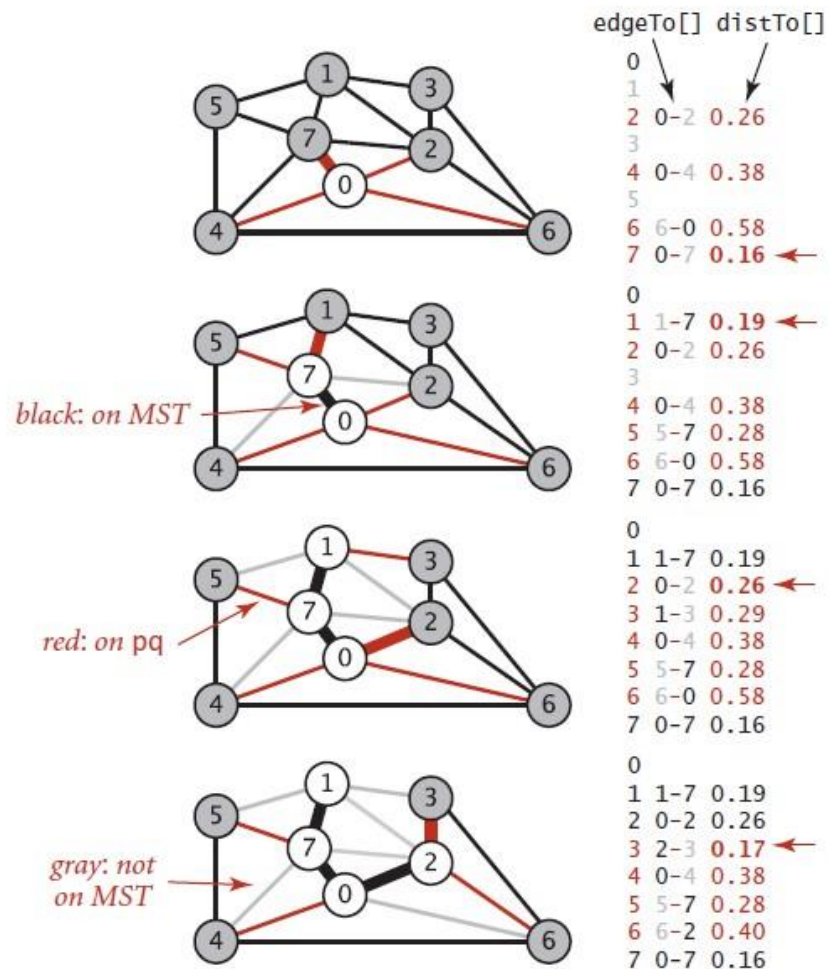
Algoritmo di Prim: versione eager

- **Eager Prim:** mantieni una PQ dei vertici (una entry per ogni vertice) connessi a T mediante almeno un arco, con priorità data dal peso dell'arco più leggero.
- delete-min vertice v e aggiungi l'arco (più corto) $e = v - w$ a T ;
- Aggiorna la PQ considerando tutti gli archi $e = v - x$
 - ignora l'arco se x è già in T ;
 - aggiungi x a PQ se non già presente;
 - decrementa la priorità di x se e diventa l'arco più corto che connette x a T .

Esempio



Esempio



Indexed priority queue

- E' il tipo di coda con priorità che consente di diminuire la priorità (key) di un vertice in essa inserito, a cui si accede tramite un indice.

```
public class IndexMinPQ<Key extends Comparable<Key>>

    IndexMinPQ(int N) //crea IPQ con indici 0..N-1
    void insert(int i, Key key) //ass. key all'indice i
    void decreaseKey(int i, Key key) //decr. key ass. a i
    boolean contains(int i) //indice i sta nella IPQ?
    int delMin() //rimuovi key minima e ritorna il suo i
    boolean isEmpty()
    int size()
```

Implementazione

```
public class PrimMST {

    private Edge[] edgeTo; // edgeTo[v] = shortest edge from tree vertex to non-tree vertex
    private double[] distTo; // distTo[v] = weight of shortest such edge
    private boolean[] marked; // marked[v] = true if v on tree, false otherwise
    private IndexMinPQ<Double> pq;

    public PrimMST(EdgeWeightedGraph G) {
        edgeTo = new Edge[G.V()];
        distTo = new double[G.V()];
        marked = new boolean[G.V()];
        pq = new IndexMinPQ<Double>(G.V());
        for (int v = 0; v < G.V(); v++)
            distTo[v] = Double.POSITIVE_INFINITY;

        for (int v = 0; v < G.V(); v++) // run from each vertex to find
            if (!marked[v]) prim(G, v); // minimum spanning forest
    }

    // run Prim's algorithm
    private void prim(EdgeWeightedGraph G, int s) {
        distTo[s] = 0.0;
        pq.insert(s, distTo[s]);
        while (!pq.isEmpty()) {
            int v = pq.delMin();
            scan(G, v);
        }
    }
    ...
}
```


Implementazione

```
...
private void scan(EdgeWeightedGraph G, int v) {
    marked[v] = true;
    for (Edge e : G.adj(v)) {
        int w = e.other(v);
        if (marked[w]) continue;           // v-w is obsolete edge
        if (e.weight() < distTo[w]) {
            distTo[w] = e.weight();
            edgeTo[w] = e;
            if (pq.contains(w)) pq.decreaseKey(w, distTo[w]);
            else pq.insert(w, distTo[w]);
        }
    }
}

public Iterable<Edge> edges() {
    Queue<Edge> mst = new Queue<Edge>();
    for (int v = 0; v < edgeTo.length; v++) {
        Edge e = edgeTo[v];
        if (e != null) {
            mst.enqueue(e);
        }
    }
    return mst;
}

public double weight() {
    double weight = 0.0;
    for (Edge e : edges())
        weight += e.weight();
    return weight;
}
```

Eager Prim: complessità

- TEOREMA: L'algoritmo di Prim nell'implementazione eager trova il MST in tempo proporzionale a $E \log V$ (caso peggiore), con spazio proporzionale a V .

- Dimostrazione: L'algoritmo usa V operazioni di *insert*, V *delete the minimum*, e (nel caso peggiore) E *cambio di priorità*.

Considerato il fatto che si usa un'implementazione heap-based di una coda con priorità indicizzata, tutte queste operazioni hanno un costo proporzionale a $\log V$.